

Using PSoC Edge, DEEPCRAFT, and YOLO for machine vision of water counter

Application note

Versions

Version	Date	Rationale
1.0	June 26, 2026	First release. Author: ROJ, KOA

Legal disclaimer

The evaluation board is for testing purposes only and, because it has limited functions and limited resilience, is not suitable for permanent use under real conditions. If the evaluation board is nevertheless used under real conditions, this is done at one's responsibility;
any liability of Rutronik is insofar excluded.

Table of contents

Introduction	4
Required hardware and software	5
Hardware	5
Software	5
Usage of DEEPCRAFT Studio	5
Data acquisition and labelling	6
Data collection	6
Data labelling	7
Data pre-processing	8
Picture filtering	8
Data augmentation	9
Training	13
Using DEEPCRAFT™ Studio	13
Interpretation of results	14
Comparing the results	14
Manual training using Yolo v5	15
Deployment	16
Using DEEPCRAFT™ Studio	16
Using Deepcraft™ Model Converter	18
Inference	19
Format input data	19
Handle output data	20
Memory usage	24
Memory overview	24
Model and working memory	24
How to assign a buffer to a specific memory	25
Impact on inference time	25

Introduction

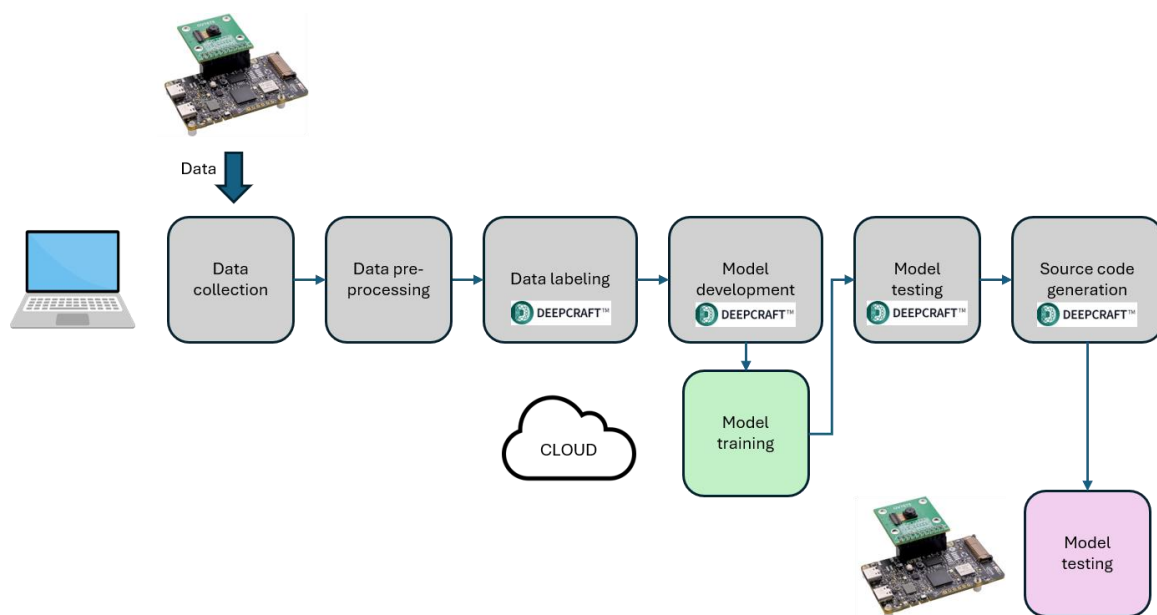
This application note describes how to recognize digits out of a video stream using machine vision on a microcontroller. The software identifies the digits out of the video stream and marks them (see the green boxes on the picture below).



The machine vision is performed by YOLO, the model is developed in DEEPCRAFT and runs on Infineon PSoC Edge MCU.

The app note covers the major steps for a machine vision project:

1. Data acquisition and data labeling.
2. Model training (YOLO v5 legacy and v5n v8n).
3. Model deployment (internal and external memory of PSoC Edge MCU).

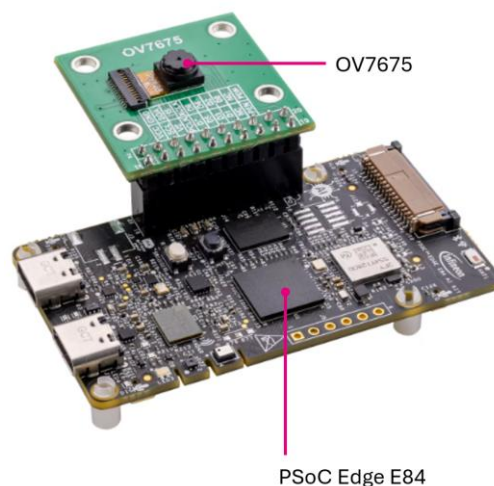


Required hardware and software

Hardware

We used Infineon PSOC™ Edge E84 AI evaluation kit. It has the OV7675 camera (320x240px) that we used for data acquisition, and the PSOC EDGE E84 with the EthosU55 NPU on it to run our model.

<https://www.infineon.com/evaluation-board/kit-pse84-ai>



Software

You can find the software example on our GitHub channel. It is directly available inside Modus Toolbox. https://github.com/RutronikSystemSolutions/kit_pse84_ai_ml_yolo

Usage of DEEPCRAFT Studio

Get rid of legal/licensing problems! Date: 08 April 2026.

Source: <https://developer.imagimob.com/legal/licensing-metrics-and-fees>

2. Licensing Metrics and Service Fees

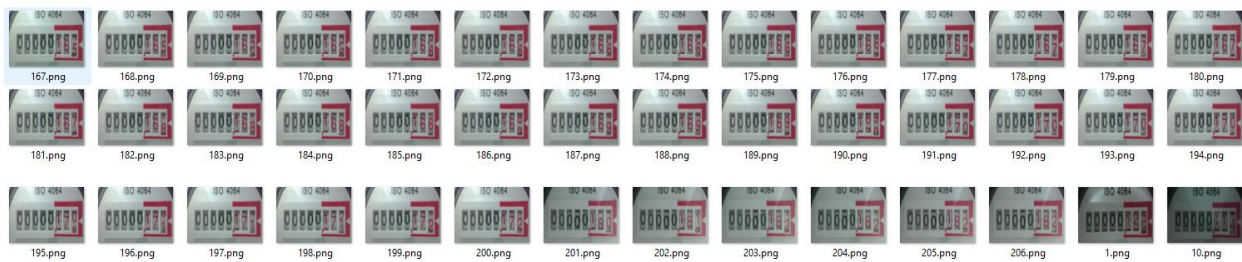
2.1 Imagimob applies the following licensing metrics and fees that Customer ("Customer" as determined in the Agreement) under the Agreement has undertaken to accept:

	Description	Fee
Use of DEEPCRAFT™ Studio including YOLO (for all MCU's)	Full access to DEEPCRAFT™ Studio for development, testing and evaluation of Edge AI models. Includes 3000 compute minutes/month of AI training. Support available through the Imagimob developer community.	Free
Use of DEEPCRAFT™ Studio or output from DEEPCRAFT Studio, including YOLO, for production and commercial deployment on Infineon MCUs and NPUs	For production and commercial deployment on Infineon MCUs and NPUs	Free

Data acquisition and labelling

Data collection

The first step is to collect data. In our case, we need pictures of a water counter with different values. We collected 344 pictures, the file explorer after collection is shown below.



We “manually” changed the value of the water counter by letting water flow in it to get different values. First 4 digits of all the pictures collected are 0 (we did not want to spend too much water).



During the training phase, we will have to pay special attention to it, because of overfitting. We do not want the model to only recognize 0, but also the other digits.

The more “diverse” the data are, the better the accuracy of the model will be. Use diverse lightning, camera distance, and other picture parameters.

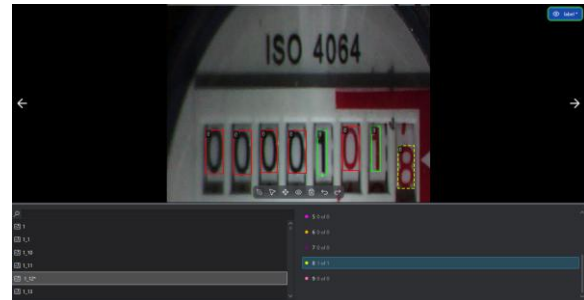
Data labelling

Labelling means that we need to add context or information to train the model. The model needs to “know” what the content of the picture is (i.e. which digits, and the position of the digits inside the picture).

You can perform the data labelling step inside DEEPCRAFT™ Studio.

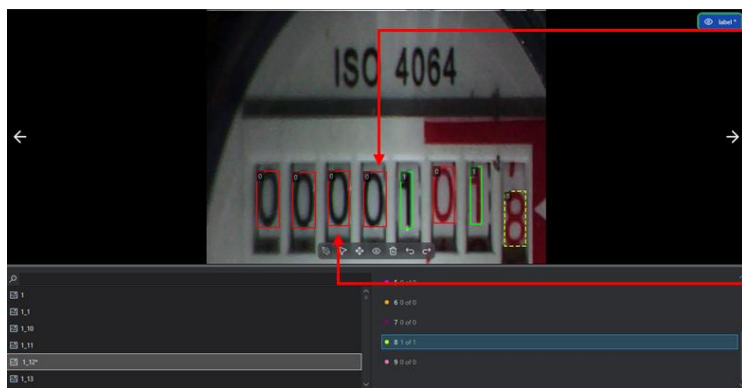


Before labeling



After labeling

You need to do this step for all collected pictures. If you use DEEPCRAFT™ Studio, the software generates an .xml file for each picture, that contains the coordinates and the classes names of the elements inside the picture.



```
<?xml version="1.0" encoding="UTF-8"?>
<Annotations>
  <Annotation>
    <Object>
      <Name>0</Name>
      <Bndbox>
        <X>0.496141975308642</X>
        <Y>0.7458847736625515</Y>
        <Width>0.06944444444444448</Width>
        <Height>0.22016460905349788</Height>
      </Bndbox>
    </Object>
  </Annotation>
  <Annotation>
    <Object>
      <Name>0</Name>
      <Bndbox>
        <X>0.38888888888888884</X>
        <Y>0.742798353909465</Y>
        <Width>0.06481481481481483</Width>
        <Height>0.21810699588477367</Height>
      </Bndbox>
    </Object>
  </Annotation>
</Annotations>
```

The units of X, Y, Width and Height is not pixel, but percentage of the picture dimensions.

Data pre-processing

Picture filtering

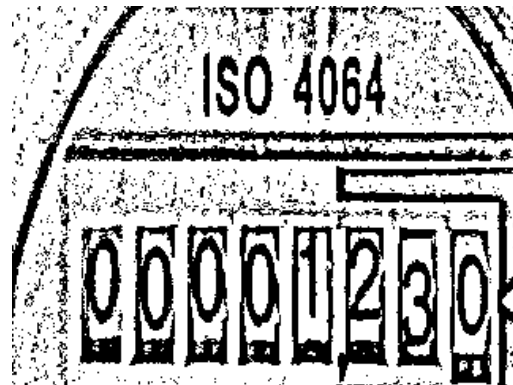
Filtering the data before the training (and the inference) can be a good solution to enhance the accuracy of the model.

In our case, we want to extract digits with their position, and we do not care about the color of those digits. As mentioned, the first 4 digits of the picture used for the training are always 0. The 3 last digits (the one in red) will have different values (since they required only a few amounts of water).

We will convert the picture in color into a black and white one.



Raw picture from the OV7675 camera



Processed picture (convert to black and white)

The processed picture will be used for the training. The script in Python to do the conversion is easy (it used OpenCV library):

```
# Load from file (img_path)
image_src = cv2.imread(img_path)

# Convert to grayscale
gray = cv2.cvtColor(image_src, cv2.COLOR_BGR2GRAY)

# Apply adaptive thresholding
bin_pic = cv2.adaptiveThreshold(gray,255,cv2.ADAPTIVE_THRESH_MEAN_C,
cv2.THRESH_BINARY,11,3)

# Save to file (output_file_path)
cv2.imwrite(output_file_path, bin_pic)
```

We will also do the color to black and white conversion when doing the inference using the model to extract the digits of an unknown picture. The implementation will be done in C, since it will be implemented on the PSOC EDGE E84.

Data augmentation

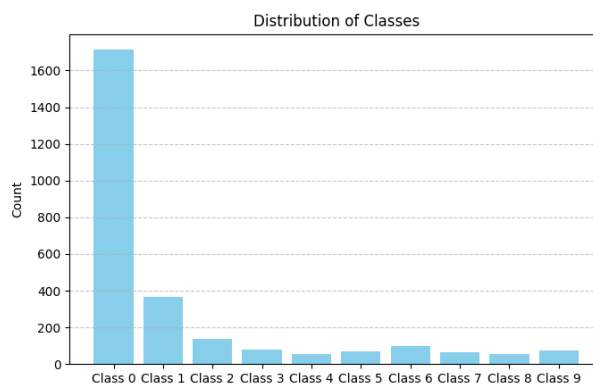
What is data augmentation?

Data labelling (the previous step) is a long process, that requires time and precision. It might also sometimes be hard to acquire a lot of diverse data.

In our case, we acquired a lot of “0” digits, but that it is hard to acquire the other digits.

With data augmentation, we will take the dataset that we acquired and labelled and generate new labelled data. Since it makes no sense to just copy the data, we will apply small transformations to those data, like rotation, scaling, adding some random pixels, ...

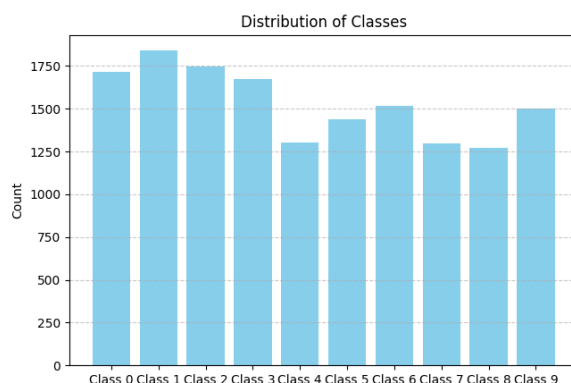
Why should we do data augmentation?



Without data augmentation, the distribution of our classes (0 to 9) is not good. Too much “0” in comparison to the other digits.

		Expected											
		unlabeled	0	1	2	3	4	5	6	7	8	9	Total
Predicted	unlabeled	0	0	0	0	36	35	43	10	34	2	2	162
	0	0	1354	0	0	0	0	0	0	0	0	0	1354
	1	21	1	296	0	0	4	0	0	1	0	0	323
	2	3	0	0	109	28	2	13	66	16	43	57	337
	3	0	0	0	0	0	0	0	0	0	0	0	0
	4	0	0	0	0	0	0	0	0	0	0	0	0
	5	0	0	0	0	0	0	0	0	0	0	0	0
	6	0	0	0	0	0	0	0	0	0	0	0	0
	7	0	0	0	0	0	0	0	0	0	0	0	0
	8	0	0	0	0	0	0	0	0	0	0	0	0
	9	0	0	0	0	0	0	0	0	0	0	0	0
Total		24	1355	296	109	64	41	56	76	51	45	59	22176

This results in a bad training (over-fitting of the “0” digit).



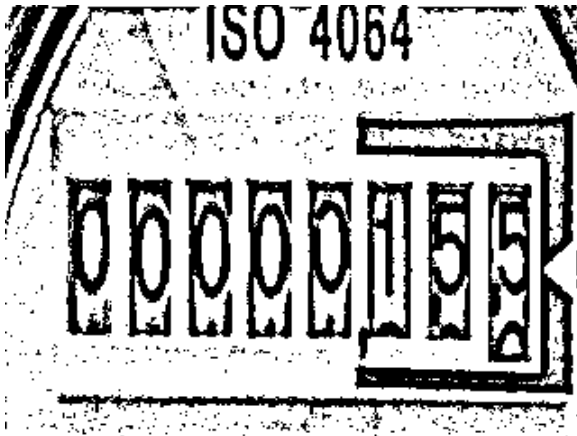
After data augmentation, the distribution of the 10 classes is much better.

		Expected											
		unlabeled	0	1	2	3	4	5	6	7	8	9	Total
Predicted	unlabeled	0	0	0	0	0	0	0	0	0	0	0	0
	0	11	1362	0	0	9	0	0	0	0	0	0	1382
	1	611	1	1480	0	0	0	0	0	0	1	0	2093
	2	538	0	1	1376	0	0	0	0	0	25	0	1940
	3	542	0	0	0	1307	0	0	0	0	1	0	1850
	4	411	0	0	0	0	1028	4	0	1	0	0	1444
	5	466	0	0	0	1	1	1117	0	0	0	0	1585
	6	503	0	1	0	0	3	14	1190	0	18	0	1729
	7	425	0	6	0	0	2	0	1	1027	0	0	1461
	8	483	0	0	0	5	0	0	13	0	937	0	1438
9	481	0	0	0	0	6	0	0	0	11	1195	1693	
Total		4471	1363	1488	1376	1328	1034	1135	1204	1028	993	1195	21661

This results in a better accuracy of our model (no overfitting of the class “0” anymore).

Data augmentation using Python scripts

Based on our actual dataset, the goal is now to generate more data for the classes 1 to 9. To do that, we will extract digits that are not 0 from the dataset and generate new pictures using it.



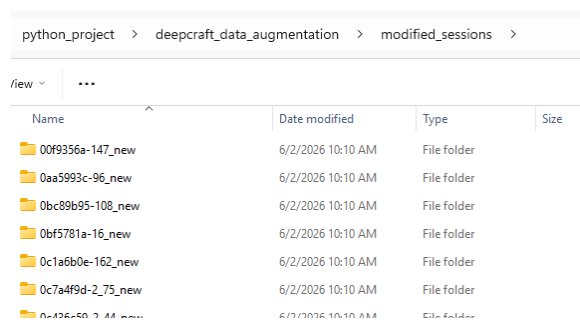
Picture from the dataset



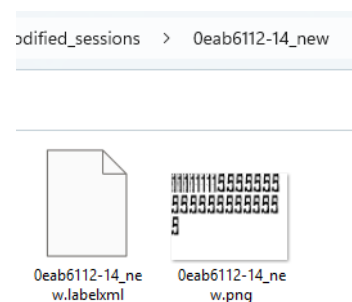
New picture generated using Python script

We will do that for all the pictures inside the dataset.

Using our Python scripts, we generated 344 new directories, that each contain as “.png” file and a “.labelxml” file:

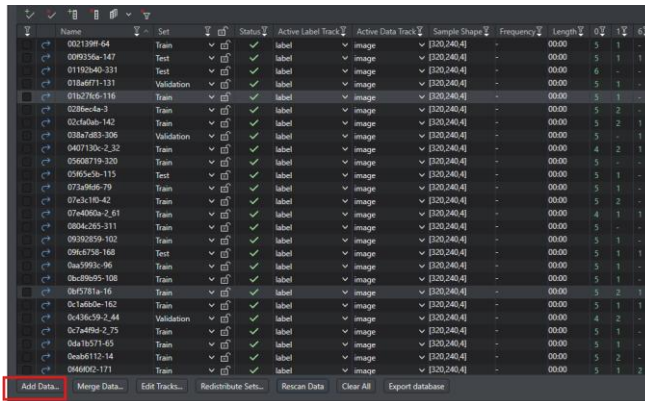


Directory containing new 344 sessions

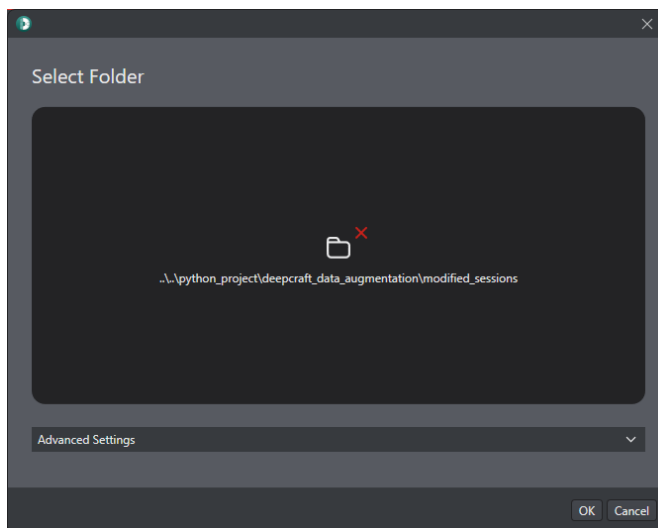


One “.labelxml” and one “.png” file per directory

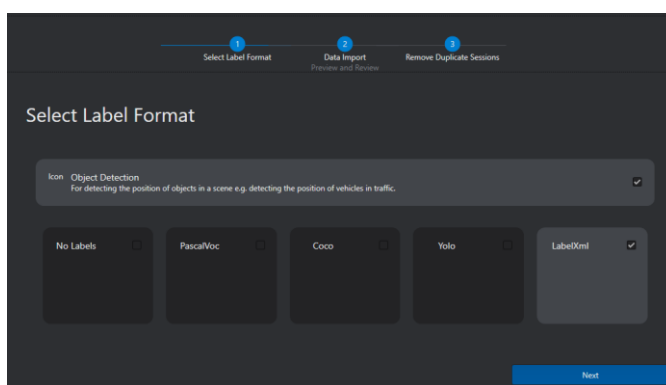
Let's add them to our project.



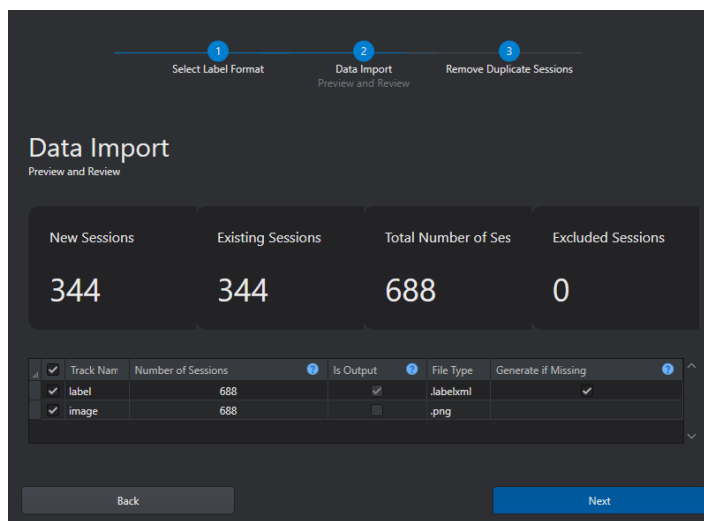
Click on the **Add Data...** button.



Select the folder to be added.



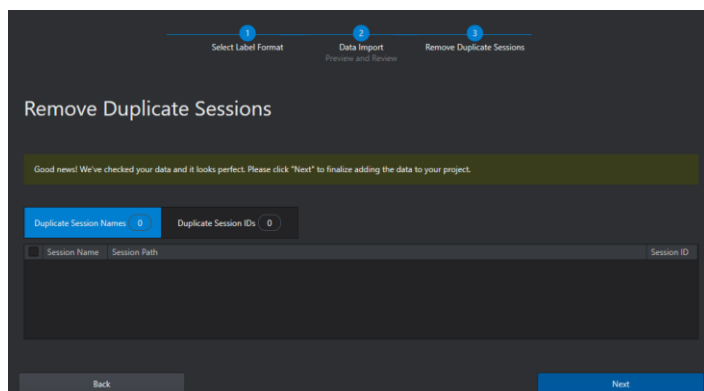
Select the format of the label file (in our case LabelXml).



A summary will be shown.

We had 344 sessions in our dataset, and we are going to add 344 new sessions.

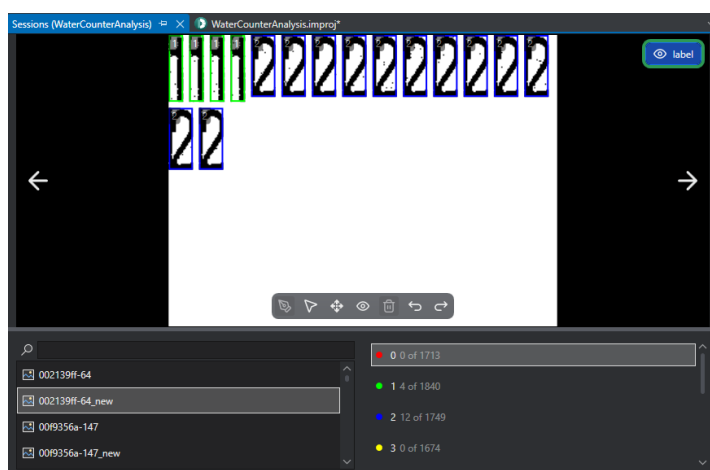
688 sessions in total.



The software will then check for duplicated sessions. If no duplicated sessions, you will see the picture on the side.

Afterwards, the new sessions will be added to the list. You can click on them to verify that they are correctly annotated.

	Name	Set	Status	Active Label Track	Active Data Track	Sample Shape	Frequency	Length	0	1	6	2	4	5	9	3	7	8
☐	002139ff-64	Train	✓	label	image	[320,240,4]	-	00:00	5	1	-	1	-	-	-	-	-	-
☐	002139ff-64_new	Test	✓	label	image	[320,240,3]	-	00:00	-	4	-	12	-	-	-	-	-	-
☐	00f9356a-147	Train	✓	label	image	[320,240,4]	-	00:00	5	1	1	1	-	-	-	-	-	-
☐	00f9356a-147_new	Train	✓	label	image	[320,240,3]	-	00:00	-	4	15	12	-	-	-	-	-	-
☐	01192b40-331	Train	✓	label	image	[320,240,4]	-	00:00	6	-	-	-	2	-	-	-	-	-
☐	01192b40-331_new	Train	✓	label	image	[320,240,3]	-	00:00	-	-	-	-	50	-	-	-	-	-

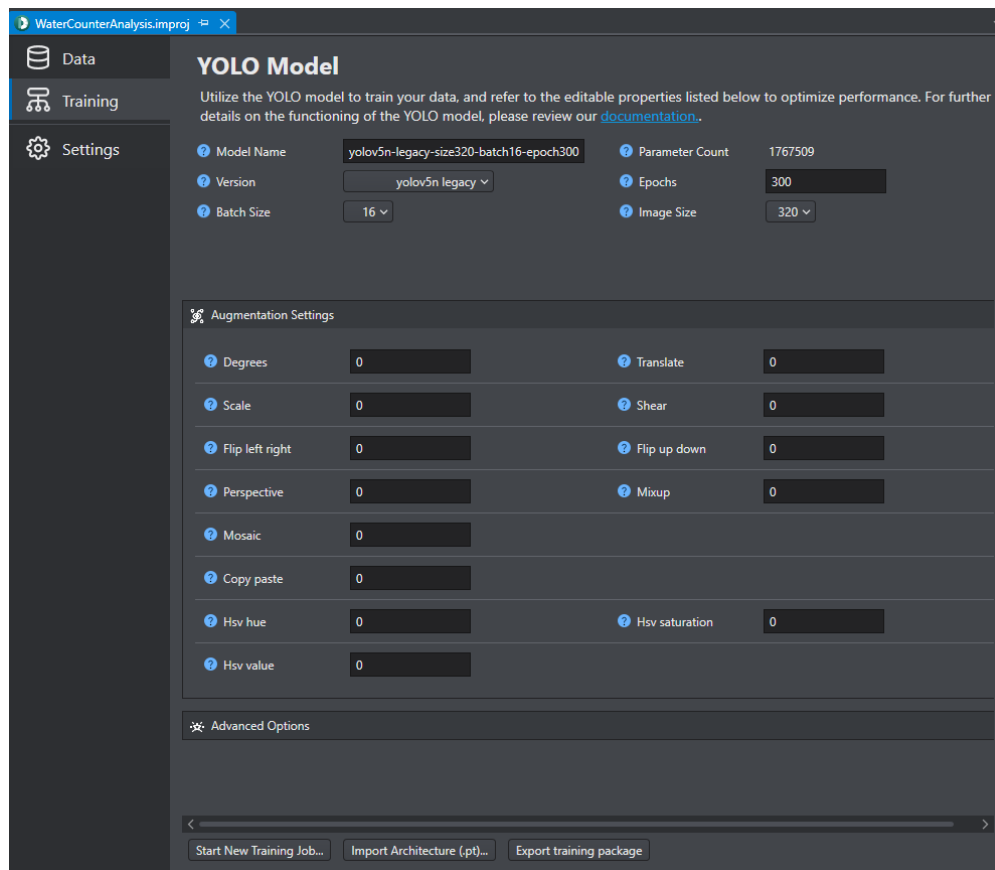


The generated file with the corresponding labels.

Training

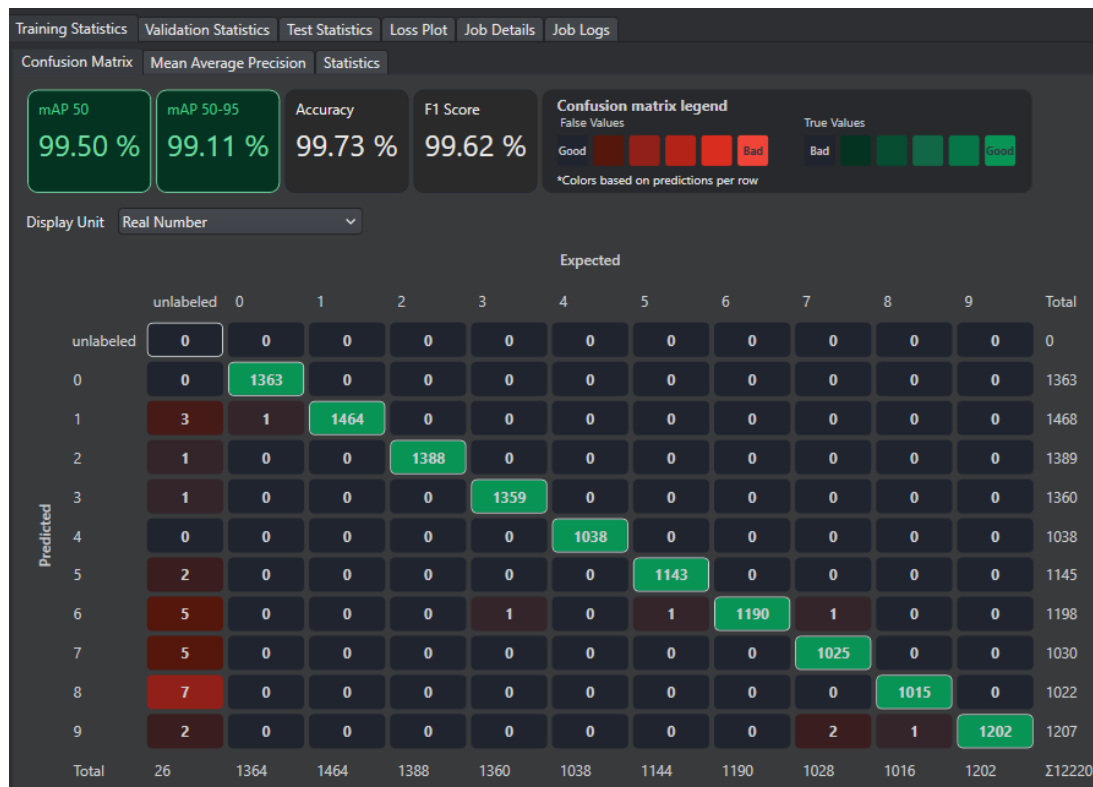
Using DEEPCRAFT™ Studio

We used version 5.12.5418.0.



Training is easy, since everything works on the DEEPCRAFT™ Studio server. You only need to adapt the settings to your needs (in my case I deactivated all the **Augmentation Settings** by setting them to 0) and click on the **Start New Training Job...**

Interpretation of results



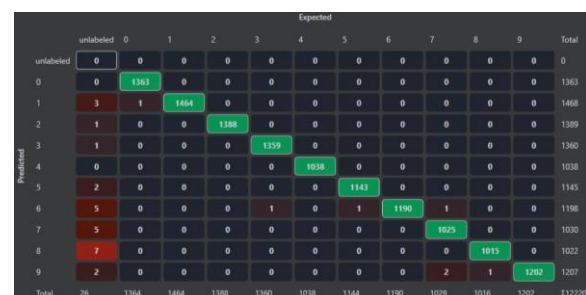
The first statistic you must look at is the confusion matrix. Use **Real Number** as **Display Unit** to have a better overview. The goal is to have the diagonal in green as above. That means the model predicts what is expected: detects a 0 as a 0 and not as a 1.

To be sure that your model behaves well on unknown data, have a look at the **Test Statistics** tabs. This confusion matrix is built using data that have not been used during the training.

Comparing the results



Model trained without data augmentation (a lot of 0) with YOLOv5n Legacy



Model trained with augmented data and YOLOv8n

The performance of the model on the right side is much better than the one on the left side. The model on the left side cannot detect the digits bigger than 3.

Manual training using Yolo v5

Follow the instructions from GitHub: <https://github.com/ultralytics/yolov5>

Our tests were done with Python 3.10.7.

```
# Clone the YOLOv5 repository
git clone https://github.com/ultralytics/yolov5

# Navigate to the cloned directory
cd yolov5

# Install required packages
pip install -r requirements.txt
```

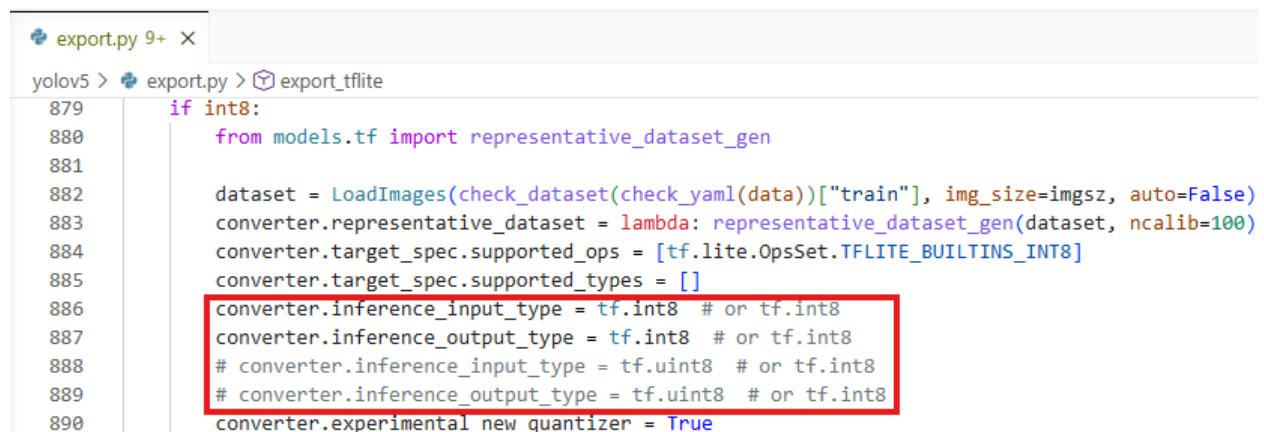
To train the model:

```
python .\yolov5\train.py --img 320 --batch 16 --epochs 100 --data .\yolo_data_bw.yaml --weights yolov5n.pt
```

You can then export it to TFLITE:

```
python .\yolov5\export.py --weights .\yolov5\runs\train\exp7\weights\best.pt --img 320 --include tflite --int8 --data .\yolo_data_bw.yaml
```

You will need to make small changes in the “export.py” script in order to be compatible with Deepcraft™ Studio Model Converter:



```
export.py 9+ X
yolov5 > export.py > export_tflite
879     if int8:
880         from models.tf import representative_dataset_gen
881
882         dataset = LoadImages(check_dataset(check_yaml(data))["train"], img_size=imgsz, auto=False)
883         converter.representative_dataset = lambda: representative_dataset_gen(dataset, ncalib=100)
884         converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
885         converter.target_spec.supported_types = []
886         converter.inference_input_type = tf.int8 # or tf.int8
887         converter.inference_output_type = tf.int8 # or tf.int8
888         # converter.inference_input_type = tf.uint8 # or tf.int8
889         # converter.inference_output_type = tf.uint8 # or tf.int8
890         converter.experimental_new_quantizer = True
```

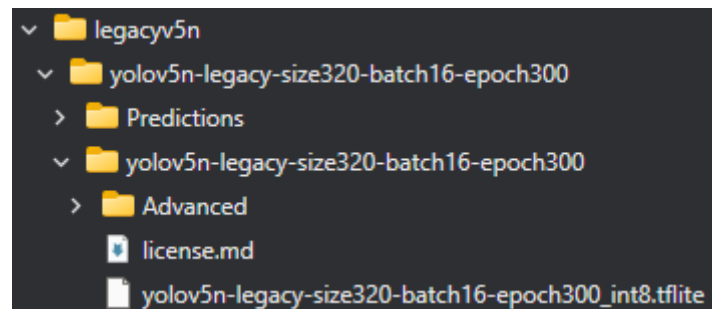
This workflow will be described in detail in the application note of ours that is currently being prepared for publication. A link to it will be added here shortly.

Deployment

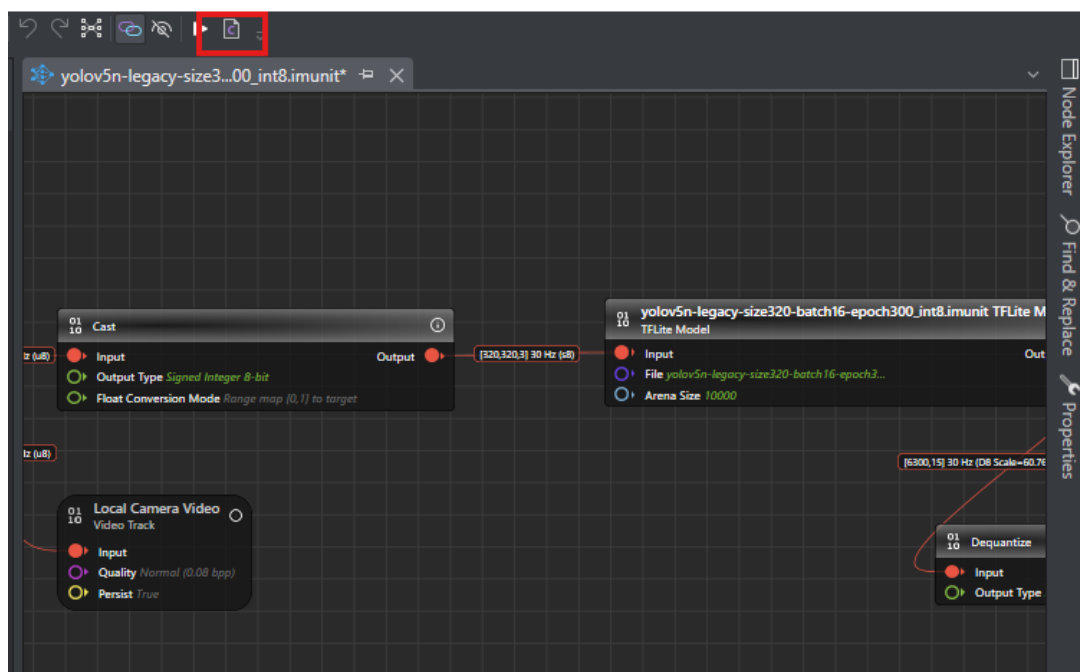
Using DEEPCRAFT™ Studio

Version used: 5.12.5418.0.

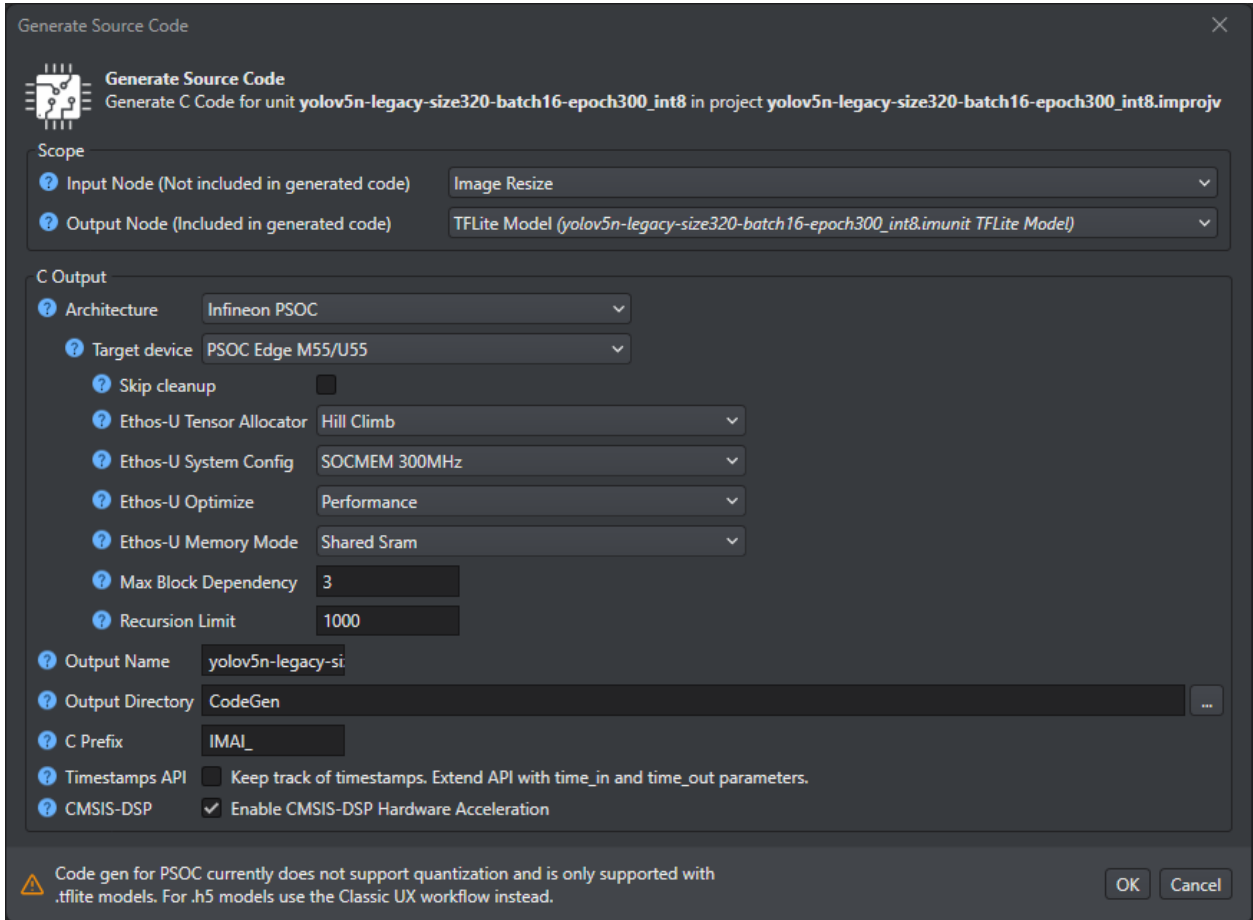
Double click on the “.tflite” model that you have downloaded.



This will create a “.imunit” graph. Click on the **C** button to generate source code:



Select following parameters and click **OK**.



Generate Source Code

Generate C Code for unit `yolov5n-legacy-size320-batch16-epoch300_int8` in project `yolov5n-legacy-size320-batch16-epoch300_int8.improjv`

Scope

Input Node (Not included in generated code) Image Resize

Output Node (Included in generated code) TFLite Model (yolov5n-legacy-size320-batch16-epoch300_int8.imunit TFLite Model)

C Output

Architecture Infineon PSOC

Target device PSOC Edge M55/U55

Skip cleanup ☐

Ethos-U Tensor Allocator Hill Climb

Ethos-U System Config SOCMEM 300MHz

Ethos-U Optimize Performance

Ethos-U Memory Mode Shared Sram

Max Block Dependency 3

Recursion Limit 1000

Output Name yolov5n-legacy-si

Output Directory CodeGen

C Prefix IMAI_

Timestamps API ☐ Keep track of timestamps. Extend API with time_in and time_out parameters.

CMSIS-DSP ☒ Enable CMSIS-DSP Hardware Acceleration

Code gen for PSOC currently does not support quantization and is only supported with .tflite models. For .h5 models use the Classic UX workflow instead.

OK **Cancel**

This will produce two files (header and source) containing everything needed to run on the PSOC Edge.

Exported methods (see in the header file):

```
// Exported methods
int IMAI_compute(const uint8_t *restrict datain, int8_t *restrict dataout);
void IMAI_finalize(void);
int IMAI_init(void);
```

The IMAI_compute method looks like:

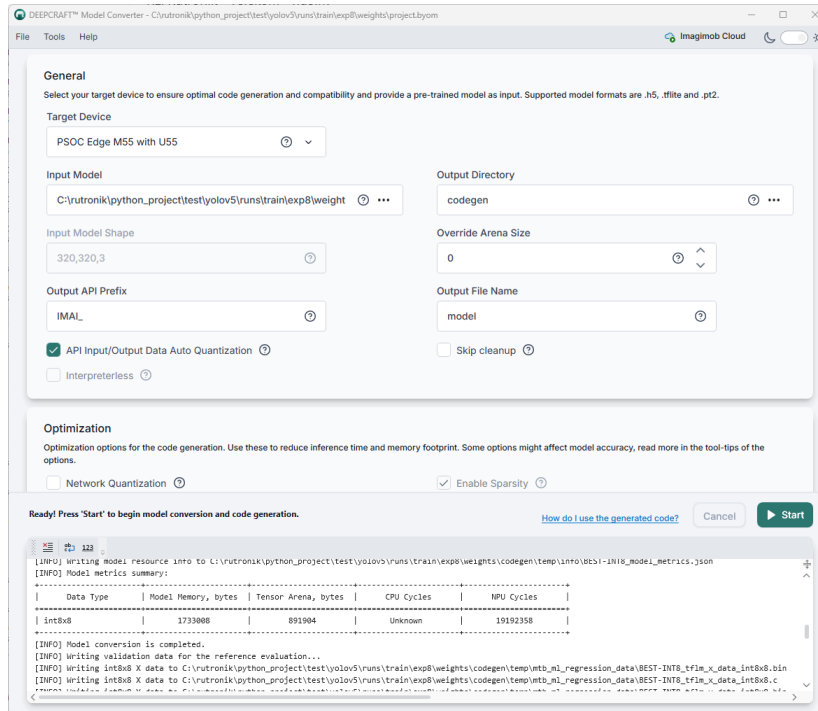
```
/*
 * @param datain Input features. Input uint8_t[320,320,3].
 * @param dataout Output tensor containing dequantized values.
 * Has the same shape as the input but with the selected output data type.
 * Output int8_t[6300,15].
 * @return IPWIN_RET_SUCCESS (0) or IPWIN_RET_NODATA (-1), IPWIN_RET_ERROR (-2), IPWIN_RET_STREAMEND (-3)
 */
int IMAI_compute(const uint8_t *restrict datain, int8_t *restrict dataout) {
    cast_u8_to_i8(datain, _K1, 307200);
    __RETURN_ERROR(mtb_model_raw(_K6, _K1, 307200, _K5, 94500));
    dequantize_d8_to_i8(_K5, dataout, 94500, 0.016458073630928993, -128);
    return 0;
}
```

Data are first converted into int8 and then the inference is started.

Using Deepcraft™ Model Converter

Version used: 5.8.4154.65534

If you trained your model using another way (Python scripts for example), you will have the possibility to convert your “tflite” model into usable source code using the DEEPCRAFT™ Model Converter tool.



The exported methods now look like:

```
// Exported methods
void IMAI_compute(const float *restrict data_in, float *restrict data_out);
void IMAI_finalize(void);
int IMAI_init(void);
```

The IMAI_compute method looks like:

```
/*
 * @param data_in Input features. Input float[320,320,3].
 * @param data_out Output features. Output float[6300,15].
 */
void IMAI_compute(const float *restrict data_in, float *restrict data_out) {
    __HOOK_REGION(true, 0);
    mtb_model_f32_to_quantized_int8(_K5, data_in, 307200, _K4, 94500, _K3);
    __HOOK_REGION(false, 0);
    __HOOK_REGION(true, 1);
    dequantize_d8_to_f32(_K4, data_out, 94500, 0.01538478210568428, -127);
    __HOOK_REGION(false, 1);
}
```

The float data are first converted to int8 (quantized), the inference runs and then the result is converted back to float (dequantized).

This workflow will be described in detail in the application note of ours that is currently being prepared for publication. A link to it will be added here shortly.

Inference

We give data to the trained model, and without writing an algorithm (as we trained the model to do that), we obtain “intelligent” information from the model. In our case, it’s the value of the flow meter.



The Inference block expects a special format (a Tensor) and will output a special format as well, containing extracted information.

In the source code (main.c file of the cm55 project), you will see the call that performs the inference:

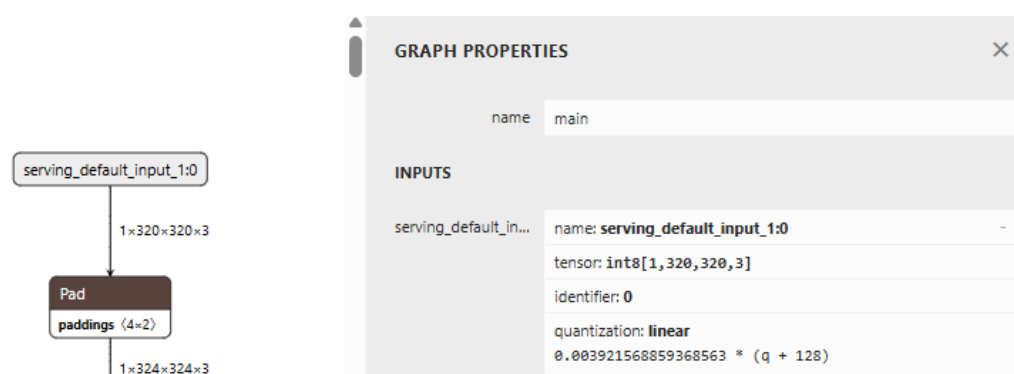
```
IMAI_compute(input_buffer, output_buffer);
```

The function expects 2 variables:

- input_buffer: A Tensor (see [“Format input data”](#)).
- output_buffer: A list of detected classes/boxes (see [“Handle output data”](#)).

Format input data

The training generates a TFLITE file, that you can open with the visualization software “Netron” (available online at netron.app).



The input of the model is a Tensor of format int8[1,320,320,3].

The code generator from Deepcraft™ Studio generates a method, depending on the used generator:

```
int IMAI_compute(
    const uint8_t *restrict datain,
    int8_t *restrict dataout);
```

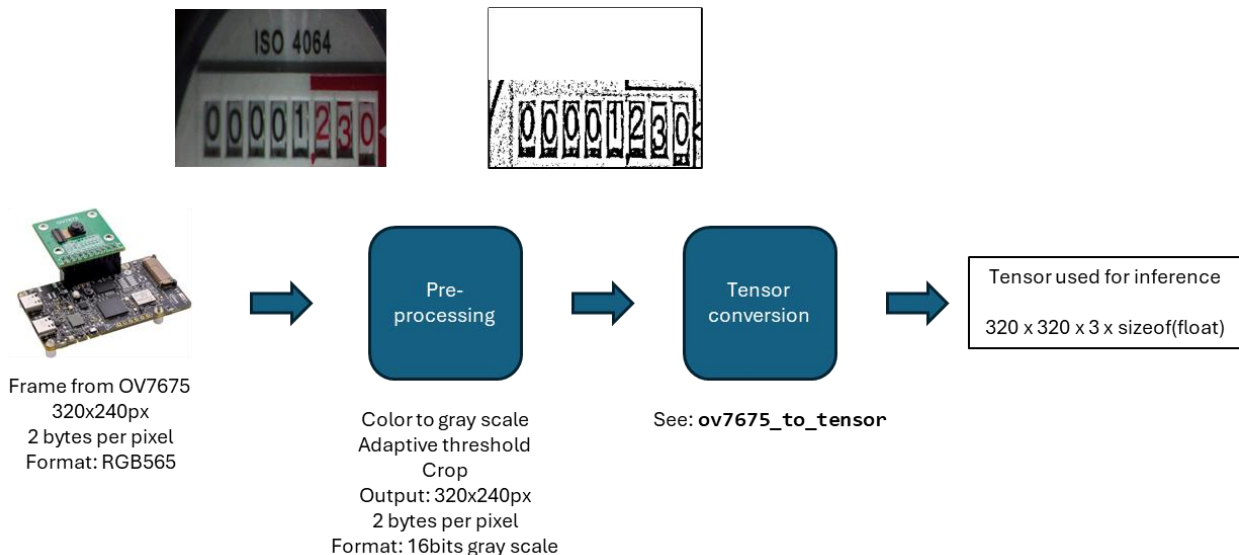
Using Deepcraft™ Studio trained model

```
void IMAI_compute(
    const float *restrict data_in,
    float *restrict data_out);
```

Using Deepcraft™ Model Converter

In case you are using Deepcraft™ Studio, the Tensor contains RGB value between 0 and 255, else, if using Deepcraft™ Model Converter, you will have to divide the RGB values by 255 (and convert them as float).

See the method `ov7675_to_tensor` in the project, to see how it is implemented.



Handle output data

OUTPUTS	
StatefulPartitione...	name: StatefulPartitionedCall:0 tensor: int8[1,6300,15] Coordinates of detected objects, class labels, and confidence score identifier: 406 quantization: linear 0.015683842822909355 * (q + 125)

Legacy v5 version

OUTPUTS	
output_0	name: PartitionedCall:0 tensor: int8[1,14,1029] identifier: 448 quantization: linear 0.0085115572437644 * (q + 118)

New version

The size of the output Tensor also depends on the size of the input Tensor, so the dimensions of the output might change depending on your setup.

Yolo v5 Legacy

OUTPUTS	
StatefulPartitio...	name: StatefulPartitionedCall:0
	tensor: int8[1,6300,15]
	Coordinates of detected objects, class labels, and confidence score
	identifier: 406
	quantization: linear
	$0.015683842822909355 * (q + 125)$

You can also find it in the generated header file:

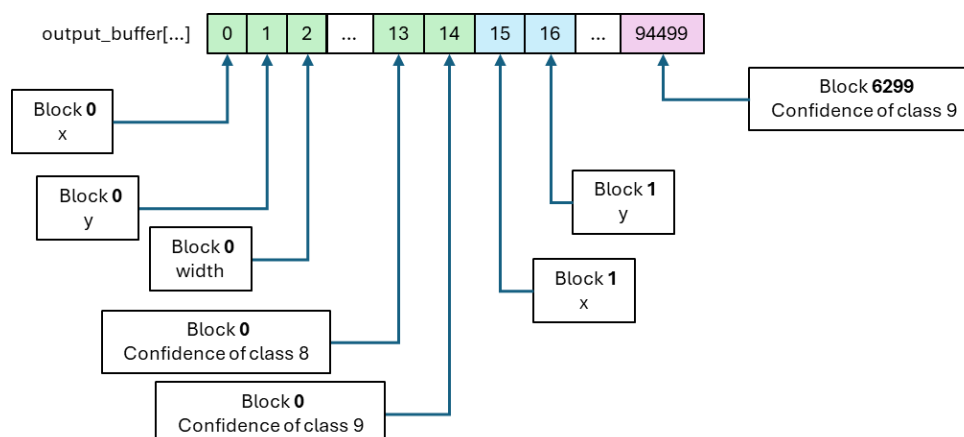
```
// dataout [6300,15] (94500 bytes)
#define IMAI_DATAOUT_SHAPE ((int[]){15, 6300})
```

Index	Meaning
0	X position (in %) of block 0
1	Y position (in %) of block 0
2	Width (in %) of block 0
3	Height (in %) of block 0
4	Block confidence of block 0
5	Confidence of class 0 (digit 0) of block 0
6	Confidence of class 1 (digit 1) of block 0
...	...
14	Confidence of class 9 (digit 9) of block 0
15	X position (in %) of block 1

```
// Defined inside generated headerfile
#define IMAI_DATAOUT_COUNT (94500)

// Inside main.c
// Allocate buffer
float * output_buffer = malloc(IMAI_DATA_OUT_COUNT * sizeof(float));

// Do inference
IMAI_compute(input_buffer, output_buffer);
```



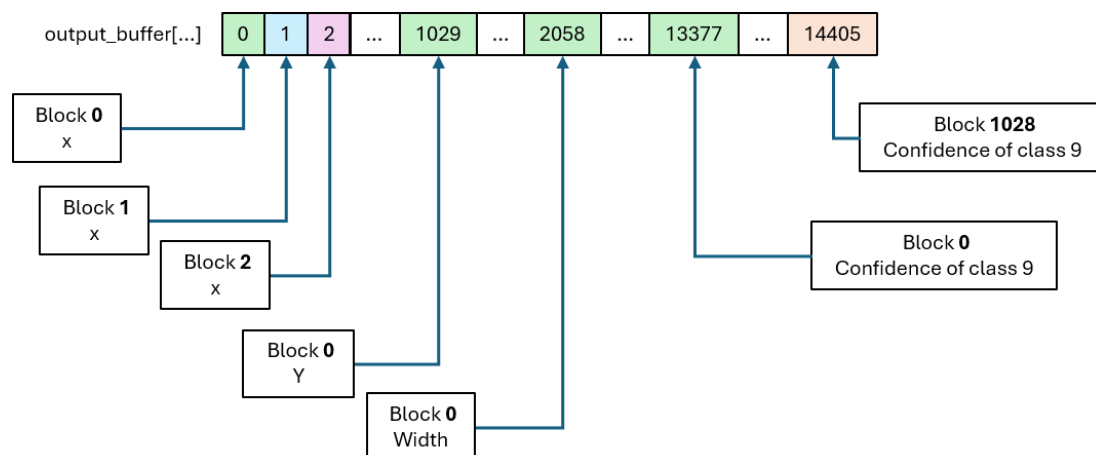
Yolo new version

OUTPUTS	
output_0	name: PartitionedCall:0
	tensor: int8[1,14,1029]
	identifier: 448
	quantization: linear
	$0.0085115572437644 * (q + 118)$

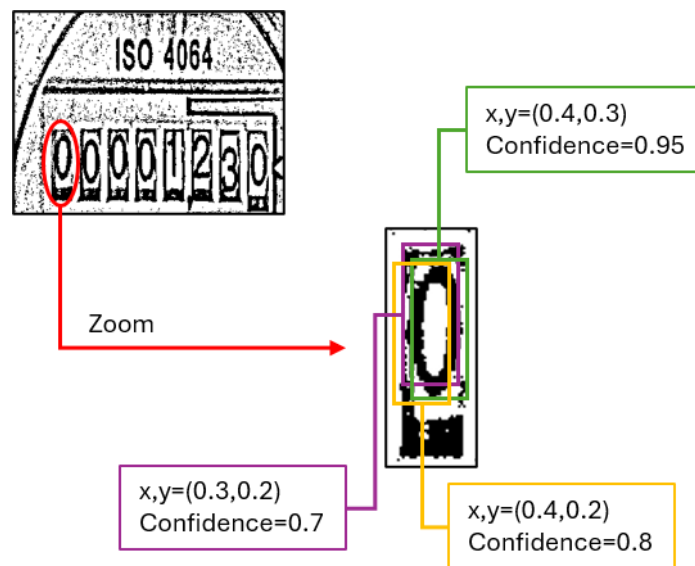
You can also find it in the generated header file:

```
// dataout [14,1029] (14406 bytes)
#define IMAI_DATAOUT_SHAPE ((int[]){1029, 14})
```

Index	Meaning
0	X position (in %) of block 0
1	X position (in %) of block 1
2	X position (in %) of block 2
...	...
1029	Y position (in %) of block 0
...	...
2058	Width (in %) of block 0
...	...
3087	Height (in %) of block 0
...	...
4116	Confidence of class 0 (digit 0) of block 0
...	...
5145	Confidence of class 1 (digit 1) of block 0
...	...
13377	Confidence of class 9 (digit 9) of block 0

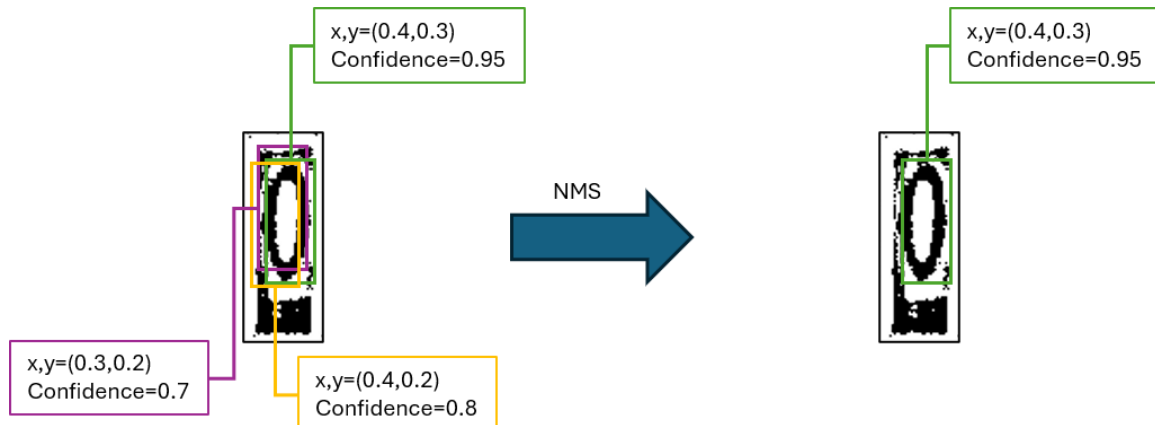


Non-maximum suppression



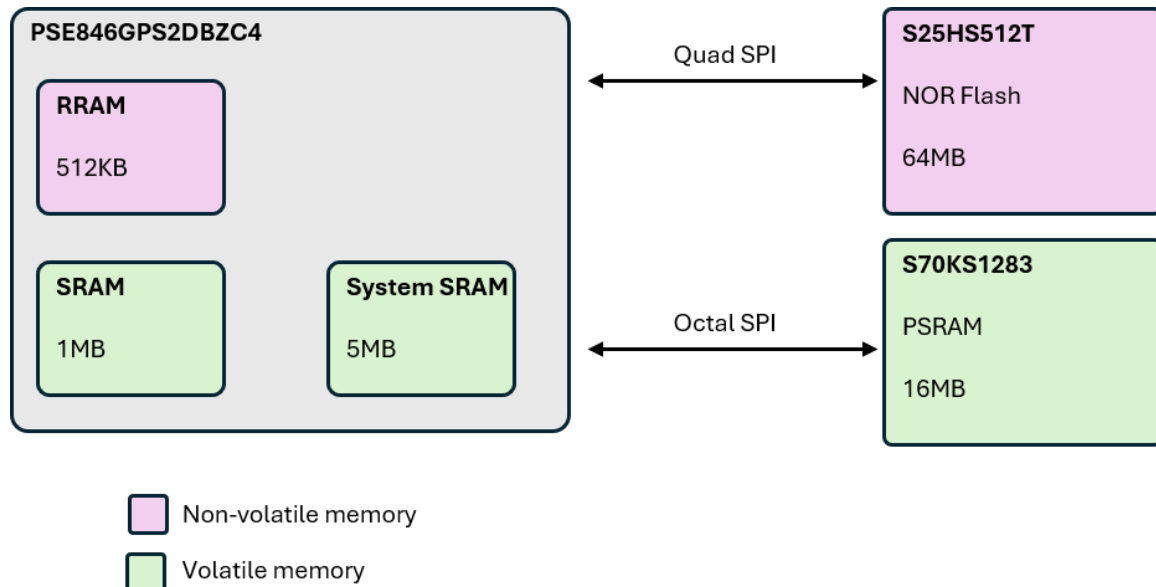
The output of the model is a list of block with confidence and coordinates. It often happens, that for one digit (in our example) the model detects blocks with high confidence that overlaps.

The NMS (non-maximum suppression) algorithm enables to only keep the block with the highest confidence.



Memory usage

Memory overview



Model and working memory

Having a look to the generated source file, you will see 3 buffers used for the inference:

Memory	Size
Buffers	401700 bytes (RAM)
State	891920 bytes (RAM)
Readonly	1688688 bytes (Flash)

Name	Description
Buffers	Contains the input layer (Tensor used to describe the picture) and the output layer (the boxes with coordinates and confidence). Stored inside RAM.
State	Buffer used during inference for computation (convolution, dense, ...). Stored inside RAM. Access to this memory should be as fast as possible. If possible, do not use an external memory but use the internal System SRAM memory.
Readonly	Store the weights of the model (computed during training). Stored inside the Flash memory, and needs to be loaded, if possible, into the internal System SRAM memory for fast access.

How to assign a buffer to a specific memory

The “CY_SECTION” usage enables to select on which memory the buffer should be stored (System SRAM, external PSRAM or external Flash).

```
#ifdef CY_ML_MODEL_MEM
#define IM_ML_MODEL_MEM CY_SECTION(EXPAND_AND_STRINGIFY(CY_ML_MODEL_MEM))
#else
#define IM_ML_MODEL_MEM
#endif

#ifdef CY_ML_ARENA_MEM
#define IM_ML_ARENA_MEM CY_SECTION(EXPAND_AND_STRINGIFY(CY_ML_ARENA_MEM))
#else
#define IM_ML_ARENA_MEM CY_SECTION(".cy_socmem_data")
#endif

#ifdef CY_ML_WORKING_MEM
#define IM_ML_WORKING_MEM CY_SECTION(EXPAND_AND_STRINGIFY(CY_ML_WORKING_MEM))
#else
#define IM_ML_WORKING_MEM CY_SECTION(".cy_socmem_data")
#endif

// Working memory
static IM_ML_WORKING_MEM ALIGNED(16) int8_t _buffer[401700];
static IM_ML_ARENA_MEM ALIGNED(16) int8_t _state[891920];

// Parameters
static IM_ML_MODEL_MEM ALIGNED(16) uint32_t _k3[] = {
```

In the Makefile, you can select where to store the buffers using DEFINES.

Example:

```
DEFINES+=CY_ML_ARENA_MEM=.cy_socmem_data
DEFINES+=CY_ML_WORKING_MEM=.cy_xip
```

- The “State” buffer used to store internal computation is stored inside the internal System SRAM (cy_socmem_data).
- The input and output layers will be stored inside the external PSRAM (cy_xip).

```
DEFINES+=CY_ML_ARENA_MEM=.cy_xip
DEFINES+=CY_ML_WORKING_MEM=.cy_xip
```

- The “State” buffer used to store internal computation is stored inside external PSRAM (cy_xip).
- The input and output layers will be stored inside the external PSRAM (cy_xip).

Impact on inference time

Memory	Size
Buffers	401700 bytes (RAM)
State	891920 bytes (RAM)
Readonly	1688688 bytes (Flash)

```
DEFINES+=CY_ML_MODEL_MEM=.cy_socmem_data
DEFINES+=CY_ML_ARENA_MEM=.cy_socmem_data
DEFINES+=CY_ML_WORKING_MEM=.cy_xip
```

Inference time: 55.2ms

```
DEFINES+=CY_ML_MODEL_MEM=.cy_socmem_data
DEFINES+=CY_ML_ARENA_MEM=.cy_xip
DEFINES+=CY_ML_WORKING_MEM=.cy_xip
```

Inference time: 215.3ms